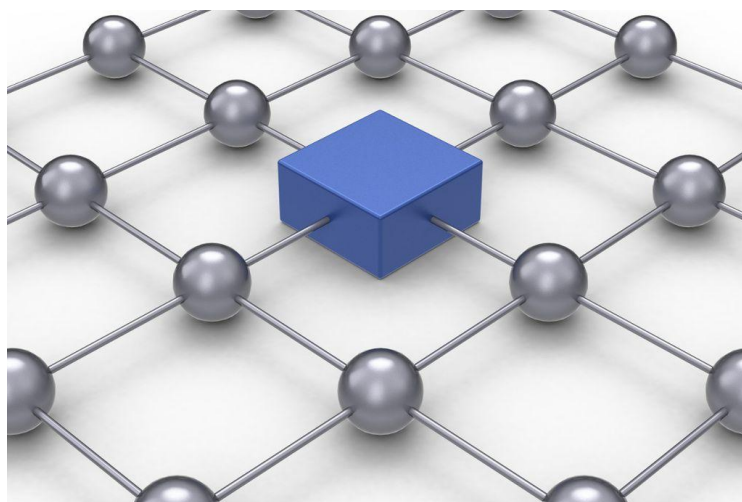




**Pełen *workflow* uczenia
maszynowego na przykładzie
rozpoznawania raka piersi**

1. Wstęp

Celem niniejszych zajęć laboratoryjnych jest wprowadzenie uczniów w podstawy sztucznej inteligencji na przykładzie klasyfikacji medycznej z wykorzystaniem sieci neuronowych. Cały proces – od pobrania danych aż po interpretację wyników – odzwierciedla rzeczywisty workflow stosowany w nowoczesnych projektach AI. Każdy z jego elementów pełni istotną funkcję i pomaga w lepszym zrozumieniu zarówno technologii, jak i praktycznych aspektów jej stosowania.

Zaczynamy od przygotowania środowiska i danych. Zrozumienie, jak dane są przekształcane i skalowane, jest kluczowe – to one stanowią paliwo dla naszych modeli. Kolejnym krokiem jest budowa sieci neuronowych w różnych wariantach – od najprostszych po bardziej zaawansowane (w tym CNN i sieci resztkowe). Dzięki temu uczniowie mogą zobaczyć, jak różne architektury wpływają na skuteczność rozwiązywania problemów.

Następnie przechodzimy do automatycznej optymalizacji – procesu, który pozwala komputerowi samodzielnie dobrać najlepsze parametry modelu. To nie tylko oszczędność czasu, ale też sposób na uzyskanie lepszych wyników bez potrzeby ręcznego eksperymentowania.

Końcowym etapem jest ocena i interpretacja wyników. Uczniowie uczą się nie tylko sprawdzać, czy model działa dobrze, ale także *dla czego* podejmuje konkretne decyzje. W tym celu wykorzystujemy m.in. metryki klasyfikacji, wykresy ROC oraz narzędzia interpretacyjne takie jak SHAP.

Dzięki takiej strukturze zajęć, uczniowie mają okazję przejść przez cały cykl projektowania systemu AI – od danych, przez model, aż po wnioski. To nie tylko nauka kodowania, ale przede wszystkim zrozumienie logiki stojącej za współczesnymi systemami inteligentnymi.

2. Wstęp do sieci neuronowych

Sieci neuronowe to jedna z najważniejszych technologii współczesnej sztucznej inteligencji. Ich działanie inspirowane jest budową i sposobem działania ludzkiego mózgu. Tak jak nasz mózg składa się z neuronów połączonych ze sobą w ogromną sieć, tak samo sztuczna sieć neuronowa zbudowana jest z wielu połączonych ze sobą „sztucznych neuronów”.

Każdy taki neuron wykonuje bardzo prostą operację: odbiera dane (czyli liczby), przelicza je według swoich „wag” (czyli tego, jak bardzo ufa każdemu wejściu), sumuje i przepuszcza przez tzw. funkcję aktywacji. Wynik tej operacji przekazywany jest dalej – do kolejnych neuronów w następnej warstwie sieci. Na początku dane wchodzi do sieci przez warstwę wejściową, a po wielu przekształceniach – wychodzą z warstwy wyjściowej, która zwraca odpowiedź modelu, np. „łagodny” albo „złośliwy”.

Proces uczenia sieci polega na tym, że pokazujemy jej dane (np. cechy guza) razem z poprawną odpowiedzią (np. czy był złośliwy). Sieć dokonuje własnej przewidywanej klasyfikacji, a następnie porównuje ją z odpowiedzią prawidłową. Jeśli się pomyli – „uczy się” na błędach. Dokładniej: modyfikuje swoje wagi w taki sposób, aby przy kolejnych podobnych przykładach lepiej trafić w poprawną odpowiedź.

To, co czyni sieci neuronowe tak potężnymi, to ich zdolność do samodzielnego znajdowania zależności w danych, których człowiek często nie jest w stanie dostrzec. Uczą się nie poprzez sztywne reguły, ale przez doświadczenie – czyli analizę ogromnej liczby przykładów. Działają skutecznie tam, gdzie klasyczne algorytmy się gubią – np. w analizie obrazów, dźwięków, języka naturalnego czy danych medycznych.

Warto pamiętać: sieć neuronowa nie „rozumie” danych jak człowiek, ale potrafi bardzo skutecznie wyłapywać powtarzalne wzorce. To czyni ją narzędziem, które nie zastępuje człowieka, lecz wspiera go – szczególnie tam, gdzie ilość danych jest zbyt duża, by człowiek mógł je ręcznie przeanalizować.

3. Przygotowanie środowiska pracy

1. Zainstaluj *Pythona* (jeśli nie masz): <https://www.python.org>
2. Uruchom terminal (*cmd*, *bash*, *PowerShell* itp.)
3. Wpisz:

```
pip install pandas numpy scikit-learn tensorflow optuna shap  
matplotlib seaborn joblib
```
4. Utwórz folder projektu o nazwie *ai_laboratorium* i umieść w nim plik *data.csv*.

4. Wstępne przygotowanie danych

Na tym etapie wczytamy dane z pliku *data.csv* i odpowiednio je przygotujemy tak aby były użyteczne w procesie uczenia i walidacji naszych modeli.

1. Wczytanie danych do obiektu typu *DataFrame*
Korzystając z funkcji *read_csv()* z modułu *pandas* wczytaj dane z pliku *data.csv* do obiektu typu *DataFrame* o nazwie *data*. Następnie wyświetl pięć pierwszych wierszy utworzonego obiektu. Zapoznaj się ze strukturą wczytanych danych.
2. Analiza eksploracyjna (*EDA*)
W ramach analizy eksploracyjnej należy sprawdzić, czy w danych są braki (w poszczególnych kolumnach), przeanalizować korelacje między poszczególnymi cechami (poza kolumną *diagnosis*) oraz sprawdzić rozkład danych i występowanie danych odstających.
 - Aby sprawdzić czy w danych są braki (z podziałem na kolumny) można wykorzystać instrukcję:

```
data.isnull().sum()
```
 - Aby wyświetlić macierz korelacji między cechami w formie graficznej (*heatmapy*) należy najpierw wygenerować taką macierz w formie numerycznej za pomocą funkcji *corr()* wywołanej na rzecz obiektu *data* (ale z pominięciem kolumny *diagnosis*):

```
corr = data.loc[:, data.columns != 'diagnosis'].corr()
```

a następnie z wykorzystaniem funkcji *heatmap()* z modułu *seaborn* (alias *sns*) wygenerować gotową *heatmapę*:

```
sns.heatmap(corr, cmap='coolwarm')  
plt.show()
```

- Rozkład danych uzyskujemy wykonując histogramy dla poszczególnych cech. Przyjmujemy podział wszystkich wartości na 30 koszy:

```
data.drop('diagnosis', axis=1).hist(bins=30,figsize=(15,0))
plt.tight_layout()
plt.show()
```

3. Przygotowanie danych, które obejmuje zmianę cechy katerycznej *diagnosis* na postać numeryczną ($M \rightarrow 0, B \rightarrow 1$), usunięcie kolumn *id* i *Unnamed: 32* oraz utworzenie na bazie obiektu *data* dwóch zbiorów: zbioru obserwacji (*X*), w którym nie będzie kolumny *diagnosis* i zbioru etykiet (*y*), składającego się z jednej kolumny danych – *diagnosis*.

- Zmiana cechy katerycznej *diagnosis* na postać numeryczną może zostać dokonana za pomocą instrukcji:

```
data['diagnosis'] = data['diagnosis'].map({'M': 1, 'B': 0})
```

- Aby usunąć kolumny *id* i *Unnamed: 32* możemy wykorzystać funkcję *drop()* wywołaną na rzecz obiektu *data*:

```
data = data.drop(['id', 'Unnamed: 32'], axis=1, errors='ignore')
```

- W celu utworzenia zbiorów *X* i *y* możemy wykorzystać odpowiednie indeksowanie obiektu *data*:

```
X = data.drop('diagnosis', axis=1).values
```

```
y = data['diagnosis'].values
```

Po wykonaniu powyższych czynności wyświetl tablice *X* i *y* oraz ich kształt.

4. Skalowanie danych dokonywane osobno dla każdej cechy według zależności:

$$x' = \frac{x - \mu}{\sigma}$$

gdzie:

x – oryginalna wartość danej cechy

μ – średnia wartość tej cechy w zbiorze

σ – odchylenie standardowe tej cechy

x' – przeskalowana (standaryzowana) wartość tej cechy

W wyniku skalowania uzyskujemy rozkład każdej cechy o średniej arytmetycznej równej w przybliżeniu 0 i odchyleniu standardowym równym w przybliżeniu 1. To pozwala porównywać różne cechy, nawet jeśli ich pierwotne skale były zupełnie inne.

Aby dokonać skalowania należy najpierw utworzyć obiekt klasy *StandardScaler()* z modułu *sklearn.preprocessing*:

```
scaler = StandardScaler()
```

i następnie dokonać operacji skalowania:

```
X_scaled = scaler.fit_transform(X)
```

Po przeprowadzeniu skalowania sprawdź, czy rzeczywiście uzyskany zbiór *X_scaled* posiada wymienione wyżej właściwości.

5. Podział danych na trzy zbiory: *treningowy*, *testowy* i *walidacyjny*.

Zbiór treningowy to podstawowy zestaw danych używany do uczenia modelu. Sieć neuronowa analizuje ten zbiór, porównuje swoje przewidywania z rzeczywistymi wynikami i dostosowuje swoje wewnętrzne parametry (tzw. wagi), aby poprawić swoje prognozy.

Zbiór walidacyjny nie służy do uczenia modelu, ale do monitorowania jego jakości podczas treningu. Model jest testowany na zbiorze walidacyjnym po każdej iteracji (epoce), aby sprawdzić, czy nie zaczyna się przeuczać (czyli zapamiętywać dane zamiast się uczyć).

Zbiór testowy używany jest tylko raz – na samym końcu. Nie wpływa na proces uczenia ani walidacji. Służy do sprawdzenia, jak dobrze wytrenowany model poradzi sobie z zupełnie nowymi, niewidzianymi wcześniej danymi.

Aby utworzyć wymienione wyżej zbiory można najpierw podzielić dane na zbiory: *trenujący* i *tymczasowy* z wykorzystaniem funkcji `train_test_split()` z modułu `sklearn.model_selection`.

```
X_train, X_temp, y_train, y_temp = train_test_split(X_scaled, y,
test_size=0.3, random_state=42)
```

a następnie podzielić zbiór *tymczasowy* na zbiory: *walidacyjny* i *testowy*:

```
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
test_size=0.5, random_state=42)
```

Wyświetl uzyskane zbiory oraz ich kształty w konsoli.

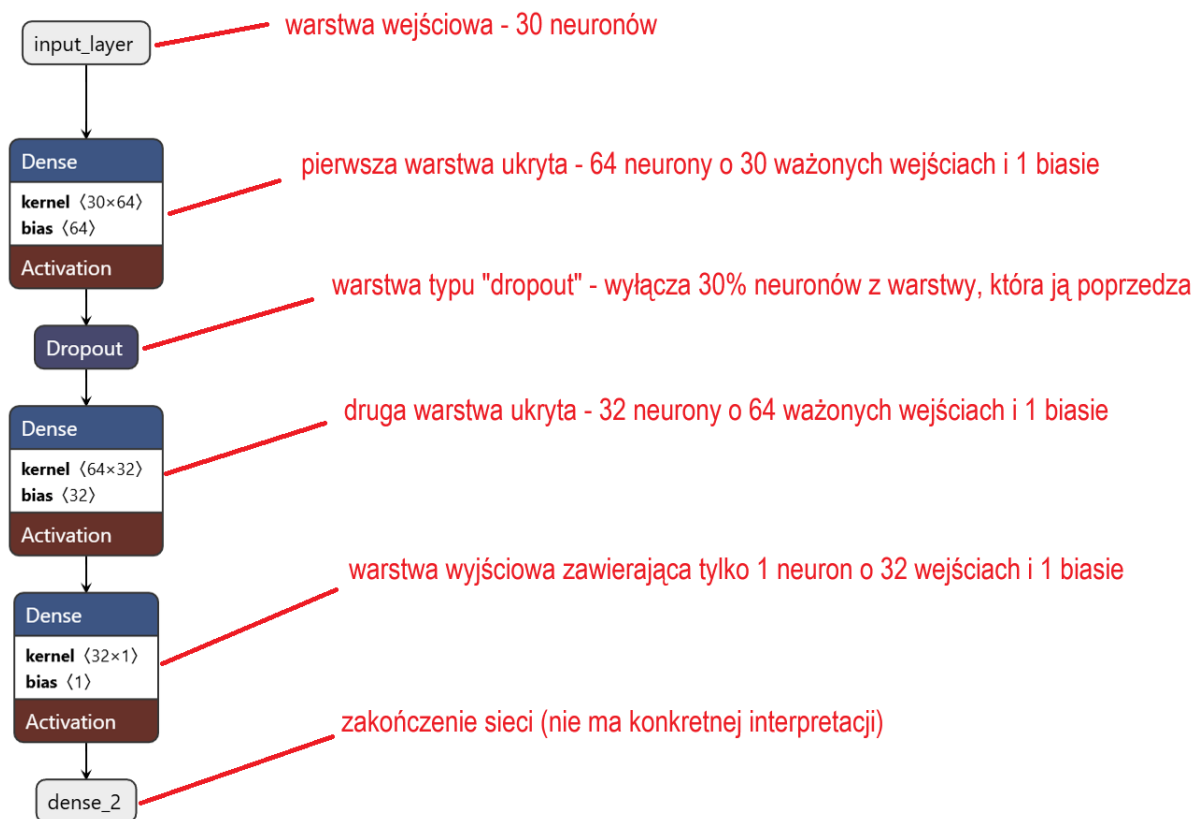
6. Tworzenie prostego modelu sekwencyjnego z wykorzystaniem klasy `Sequential()`

W tej części należy stworzyć prosty model sekwencyjny składający się z warstwy wejściowej typu *Input*, dwóch warstw ukrytych typu *Dense* oraz jednej warstwy wyjściowej typu *Dense*. Na następnej stronie przedstawiono graficzną wizualizację tego modelu wraz z dokładnym opisem poszczególnych warstw.

Aby utworzyć taką architekturę sieci należy wykonać następujące czynności:

- Utworzyć model klasy `Sequential()` z modułu `tensorflow.keras.models`:
`model = Sequential()`
- Dodać kolejne warstwy do modelu za pomocą funkcji `add()` wywołanej na rzecz obiektu *model*:
`model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))`
`model.add(Dropout(0.3))`
`model.add(Dense(32, activation='relu'))`
`model.add(Dense(1, activation='sigmoid'))`
- Wyświetlić informacje na temat tego modelu, aby zweryfikować czy jego struktura odpowiada założeniom:
`model.summary()`

Warto przeanalizować tabelkę zwróconą przez metodę *summary()* – w szczególności warto upewnić się, że wiemy dlaczego trenowanych jest tyle a nie inna ilość parametrów.



Rysunek 1. Model sieci do zadania 6.

Następnie należy skompilować utworzony model z wykorzystaniem metody *compile()* wywołanej na rzecz obiektu *model()*:

```
model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])
```

Poniżej wyjaśniono znaczenie poszczególnych parametrów tej metody oraz wartości, które przyjmują:

optimizer - algorytm, który steruje tym, jak model uczy się podczas treningu – czyli jak aktualizuje swoje wagi na podstawie błędów

Adam – rodzaj algorytmu optymalizacyjnego; bierze pod uwagę „pęd” poprzednich zmian i dostosowuje krok uczenia osobno dla każdej wagi; szybko się uczy i dobrze sprawdza w praktyce, zwłaszcza przy dużych zbiorach i sieciach

learning_rate - to krok uczenia się decydujący o tym, jak duża ma być korekta wag przy każdej aktualizacji; jeśli *learning_rate* jest za duży – sieć może „przeskakiwać” dobre rozwiązania i nigdy nie osiągnąć optimum, gdy za mały – sieć uczy się zbyt wolno (może „utknąć” w miejscu)

loss – to funkcja straty; „mówi” modelowi jak bardzo się myli

binary_crossentropy – to funkcja odpowiednia dla klasyfikacji binarnej (czyli dwóch klas, np. 0 vs 1); działa na podstawie rozbieżności między prawdziwą etykietą a przewidywanym prawdopodobieństwem

metrics – lista metryk obliczanych w trakcie procesu uczenia; w tym przypadku mierzona jest dokładność modelu (*accuracy*)

Uczenie modelu polega na podawaniu kolejnych przykładów uczących (obserwacji wraz z ich etykietami). Wagi modelu są aktualizowane po podaniu określonej porcji przykładów uczących (*batch*). Podanie wszystkich przykładów uczących to jedna *epoka*. Uczenie może składać się z wielu epok. Ponadto ponieważ w trakcie procesu uczenia chcemy dokonywać walidacji jego aktualnego działania, należy podać również zbiór danych walidacyjnych. Aby rozpocząć uczenie modelu należy na jego rzecz wywołać funkcję *fit* i przekazać dane trenujące, walidacyjne, określić liczbę epok oraz rozmiar pojedynczego *batch’a*. Przedstawione poniżej wywołanie tej funkcji uczy model *model* w 30 epokach, przy czym aktualizacja wag następuje po podaniu 16 przykładów uczących.

```
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),  
epochs=30, batch_size=16, verbose=1)
```

Uwaga:

- Zapis wyniku do zmiennej *history* jest opcjonalne i pozwala na późniejsze prześledzenie procesu uczenia (*postprocessing*)
- Parametr *verbose* ustawiony na 1 pozwala na podgląd efektów uczenia w konsoli w czasie rzeczywistym

Na samym końcu należy dokonać ewaluacji modelu na zbiorze testowym:

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
```

Dokładność modelu dla zbioru testowego zostanie zapisana w zmiennej *test_acc*.

Aby wykonać pojedynczą predykcję, tzn. przewidzieć klasę 1 lub 0 dla wybranego przykładu uczącego należy wywołać na rzecz modelu metodę *predict()* – zostanie zwrócone prawdopodobieństwo przynależności do klasy 1, które musimy samodzielnie zinterpretować jako klasę 1 albo 0:

```
model.predict(X_train[0].reshape(1, -1))
```

Uwaga:

Potrzeba zmiany kształtu obserwacji wynika z faktu, iż jedna obserwacja jest wektorem 1D, podczas gdy funkcja *predict()* oczekuje macierzy 2D, gdyż docelowo ma dokonywać predykcji dla wielu obserwacji naraz.

Wycudzony model można zapisać jako plik w formacie `.keras` i wyświetlić w postaci graficznej w aplikacji webowej *Netron*:

```
model.save(r"C:\Users\TK\Desktop\gry\ai_laboratorium\seq_model.keras")
```

Link do aplikacji *Netron*: <https://netron.app/>

7. Tworzenie prostego modelu sekwencyjnego wykorzystującego *API funkcjonalne*

W ramach tej części należy przygotować, skompilować i wytrenować ten sam model, co w części 6., ale z wykorzystaniem tzw. *API funkcjonalnego* modułu *keras*. Zasadnicza różnica polega na tym, że tutaj traktujemy każdą warstwę trochę jak funkcję, która jako parametr wejściowy przyjmuje warstwę poprzednią (to przekazanie warstwy jako parametr wejściowy innej warstwy stanowi *de facto* połączenie obu tych warstw)

Przykład:

Jeżeli mamy dwie warstwy *X* i *Y* to wywołanie:

Y(X) oznacza, że warstwa *X* dostaje połączona z warstwą *Y*, tzn. wyjścia warstwy *X* stają się wejściami warstwy *Y*.

Dzięki temu możemy zdefiniować każdą warstwę jako osobny obiekt:

Warstwa wejściowa:

```
wejście = Input(shape=(30,), name="wejście")
```

Warstwy ukryte

```
warstwa1 = Dense(64, activation='relu', name="ukryta1")(wejście)
```

```
dropout1 = Dropout(0.3, name="dropout1")(warstwa1)
```

```
warstwa2 = Dense(32, activation='relu', name="ukryta2")(dropout1)
```

Warstwa wyjściowa

```
wyjście = Dense(1, activation='sigmoid', name="wyjście")(warstwa2)
```

Teraz tworzymy obiekt modelu o nazwie *model* (z modułu *Model*):

```
model2 = Model(inputs=wejście, outputs=wyjście)
```

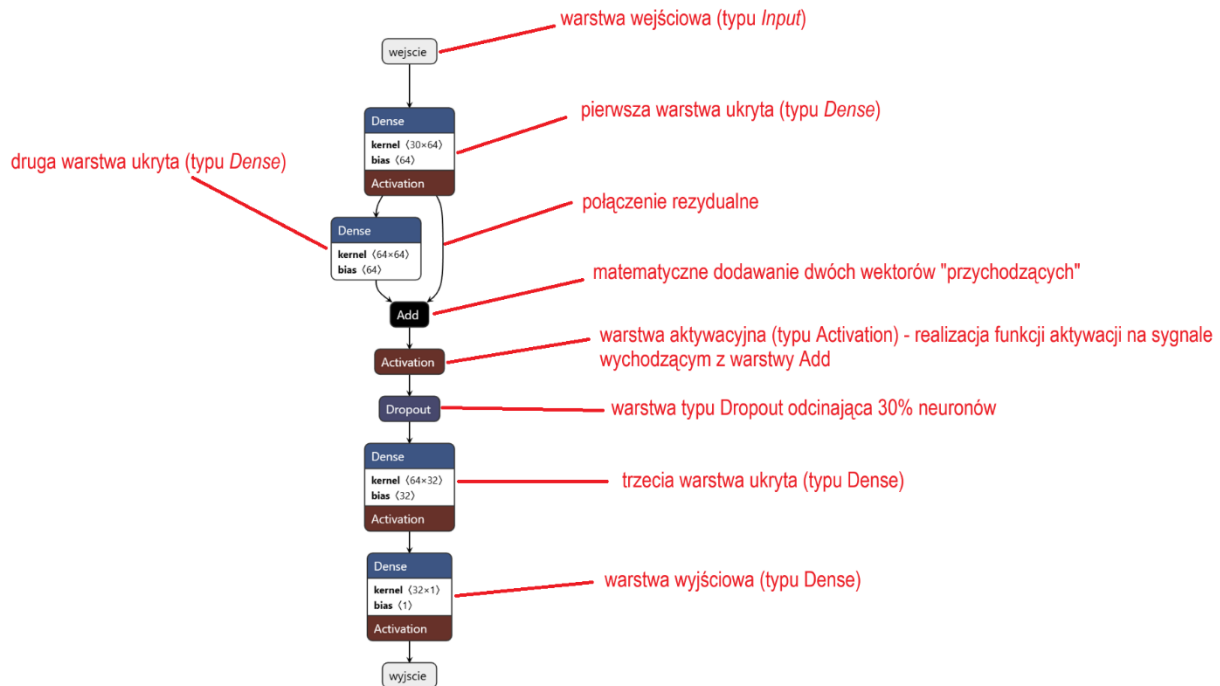
Uwaga:

Zauważmy, że aby utworzyć ten model wystarczy podać warstwę wejściową i wyjściową. Pozostałe warstwy są sekwencyjnie połączone z warstwą wyjściową.

Ponownie warto zapisać nauczony model w postaci pliku `.keras` i zerknąć na jego wizualizację graficzną w programie *Netron*.

8. Tworzenie sieci z połączeniami rezyduальnymi

Stwórz, skompiluj i wytrenuj model zgodnie z poniższym schematem. Zwróć uwagę na opisy poszczególnych warstw.



Rysunek 1. Model sieci do zadania 8.

Wskazówki implementacyjne:

- Należy w warstwach ukrytych typu *Dense* wykorzystać funkcję aktywacji *relu*
- W warstwie wyjściowej należy zastosować sigmoidalną funkcję aktywacji
- Aby utworzyć widoczne na powyższym rysunku połączenie *rezydualne* należy jako parametr warstwy typu *Add* podać listę warstw dochodzących, tzn. pierwsza i druga warstwa ukryta:

```
warstwa_add = Add(name="add_shortcut")([pierwsza_ukryta, druga_ukryta])
```

Ponownie warto zapisać nauczony model w postaci pliku `.keras` i zerknąć na jego wizualizację graficzną w programie *Netron*.

9. Tworzenie sieci opartej na modelu *CNN* (model konwolucyjny)

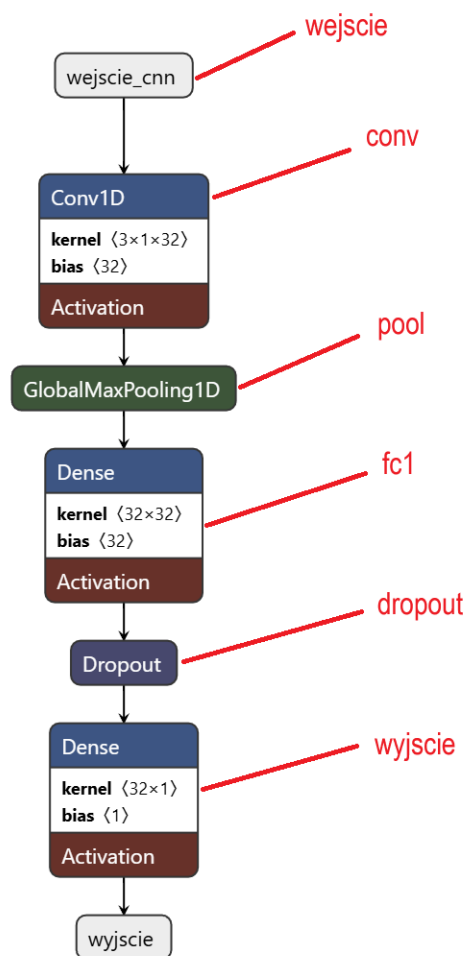
W ramach tej części zostanie utworzona sieć oparta na modelu konwolucyjnym (*CNN*). Model konwolucyjny to specjalny typ sieci neuronowej, który uczy się wykrywać wzorce lokalne w danych. Zamiast analizować całe dane naraz (jak w warstwach *Dense*), *CNN* używa filtrów przesuwających się po fragmencie danych i wyłuskuje lokalne cechy (np. "krawędź", "napiecie", "zmiana"). *CNN* przesuwają filtr (np. 3-cechowy) po danych

wejściowych, przez co uczy się on reagować na konkretne wzorce (np. nagły wzrost, nieregularność). Następnie wyniki z różnych pozycji są zbierane i dalej przetwarzane. Efekt jest taki, że sieć jest bardziej odporna na zakłócenia w danych, a przy tym bardziej zwięzła i mniej podatna na przeuczenie.

To, że sieć jest *1D* oznacza, że dane wejściowe to ciąg liczb (wektor), np. dane czasowe, sygnały biologiczne, cechy medyczne. W trakcie uczenia i wnioskowania filtr przesuwają się wzdłuż jednej osi — czyli analizuje dane cecha po cenie w małych oknach. W przeciwieństwie do *CNN 2D* — dla obrazów (filtry poruszają się w pionie i poziomie) oraz *CNN 3D* — dla wideo lub objętościowych danych (filtry poruszają się w trzech wymiarach).

W analizie danych medycznych (na przykład takich jakimi my się zajmujemy) sieć *1D* może np. nauczyć się reagować na lokalne zmiany między cechami takimi jak *radius_mean*, *texture_mean*, *perimeter_mean* itd. Dzięki temu może uchwycić lokalne wzorce biologiczne, które byłyby trudne do dostrzeżenia w klasycznym modelu gęstym.

Utworzymy sieć o poniższej architekturze. W celu zwiększenia czytelności komentarzy odnośnie poszczególnych warstw umieścimy je obok ilustracji. Na obrazku zostały umieszczone robocze nazwy poszczególnych warstw używane przy ich opisach.



Schemat sieci do zadania 9.

wejście – definiuje kształt danych wejściowych do modelu; przyjmuje dane w postaci sekwencji 30 cech z 1 kanałem (wymagane przez warstwę konwolucyjną); nie przetwarza danych — tylko ustala punkt wejścia dla dalszych warstw

conv – stosuje 32 różne filtry przesuwane po danych (okno o rozmiarze 3 cechy); każdy filtr uczy się wykrywać lokalny wzorec między sąsiednimi cechami (np. nagły wzrost, przejście); wykorzystuje funkcję aktywacji *ReLU* ($\max(0, x)$) dzięki czemu przekazuje dalej tylko dodatnie sygnały; tworzy mapy cech, które reprezentują wykryte wzorce

pool – redukuje wymiar mapy cech do jednej wartości na filtr; wybiera największy (najsilniejszy) sygnał wykryty przez każdy filtr; pomaga uprościć dane i zmniejszyć liczbę parametrów

fc1 – standardowa warstwa w pełni połączona z 32 neuronami; każdy neuron uczy się kombinacji sygnałów z poprzedniej warstwy; jej funkcja aktywacji *ReLU* wprowadza nieliniowość, umożliwiając modelowi rozpoznawanie złożonych wzorców

dropout – losowo wyłącza 30% neuronów podczas treningu; pomaga zapobiegać przeuczeniu i dodatkowo wymusza na modelu większą odporność i uogólnienie

wyjście - warstwa wyjściowa z jednym neuronem i aktywacją w postaci funkcji sigmoidalnej; zwraca wartość z przedziału (0, 1), interpretowaną jako prawdopodobieństwo klasy 1; stosowana do klasyfikacji binarnej (np. guz łagodny vs złośliwy).

Wskazówki implementacyjne:

- Aby utworzyć warstwę konwolucyjną (*conv*) należy poza nazwą funkcji aktywacji podać również liczbę filtrów i rozmiar jądra pojedynczego filtra:

```
conv = Conv1D(filters=32, kernel_size=3, activation='relu', name="conv1d")(wejście)
```

- W celu utworzenia warstwy zmniejszającej wymiarowość danych należy jedynie podać nazwę tej warstwy oraz „podpiąć” odpowiednią warstwę poprzedzającą:

```
pool = GlobalMaxPooling1D(name="global_pool")(conv)
```

Ponownie warto zapisać nauczony model w postaci pliku *.keras* i zerknąć na jego wizualizację graficzną w programie *Netron*.

10. Optymalizacja z wykorzystaniem modułu *optuna* cz. 1

Rozważamy wszystkie graniastosłupy prawidłowe trójkątne o polu powierzchni całkowitej równym $24\sqrt{3}$.

Wówczas:

Objętość V graniastosłupa w zależności od długości a krawędzi podstawy jest określona wzorem

$$V(a) = 6a - \frac{1}{8}a^3$$

dla $a \in (0, 4\sqrt{3})$.

Wyznacz długość krawędzi podstawy tego z rozważanych graniastosłupów, którego objętość jest największa. Oblicz tę największą objętość. Zapisz obliczenia.

Aby uzyskać iteracyjnie przybliżone rozwiązanie tego problemu można posłużyć się modulem *optuna*.

Na wstępie należy zdefiniować tzw. funkcję celu, tzn. funkcję, dla której będziemy poszukiwać argumentu, dla którego przyjmuje ona wartość ekstremalną w zadanym przedziale. W naszym przypadku funkcją celu jest $V(a)$, a przedziałem jest zbiór $a \in (0, 4\sqrt{3})$, co w przybliżeniu daje przedział $a \in (0; 6,93)$.

Funkcja celu powinna posiadać następujące elementy:

- powinna posiadać dokładnie jeden parametr wejściowy – najczęściej przypisuje mu się nazwę *trial*, co w dosłownym tłumaczeniu znaczy „eksperyment” – każda iteracja to jeden eksperyment, w którym przyjmujemy konkretną wartość określonego parametru (u nas a)
- Powinna posiadać wywołanie metody *suggest_**(), która w danej iteracji zwraca pewien obiekt ze zdefiniowanego przedziału. W naszym przypadku funkcja ta zwraca w każdej iteracji jedną liczbę z przedziału $(0; 6,93)$.
- Powinna zwracać wartość, dla której szukamy minimum lub maksimum.

Dla naszego przykładu funkcja celu może mieć postać jak poniżej:

```
def V(trial):
```

```
    a = trial.suggest_float('a', 0, 6.93)
```

```
    return 6 * a - 1/8 * a ** 3
```

Następnie należy utworzyć obiekt *study* za pomocą funkcji *create_study()* z modułu *optuna*, podając jako parametr wejściowy sposób optymalizacji – minimum lub maksimum:

```
study = optuna.create_study(direction="minimize")
```

W dalszej części dokonujemy optymalizacji wywołując metodę *optimize()* na bazie obiektu *study*. Jako parametry wejściowe przekazujemy referencję do funkcji celu (u nas jest to funkcja *V()*) oraz liczbę iteracji:

```
study.optimize(V, n_trials=100)
```

Argument funkcji celu, dla którego funkcja ta osiąga wartość ekstremalną, jak również sama ta wartość ekstremalna są składowymi obiektu *study* (odpowiednio *best_params* i *best_value*):

```
study.best_params
```

```
study.best_value
```

Warto zaznaczyć, że parametrów optymalizacji może być więcej niż jeden. Innymi słowy możemy optymalizować również funkcje wielu zmiennych.

11. Optymalizacja z wykorzystaniem modułu *optuna* cz. 2

W ramach tej części będziemy chcieli dobrać architekturę sieci neuronowej do rozpoznawania cyfr pisanych odręcznie. W pliku *utils.py* znajdują się pewne funkcje pomocnicze:

NHN() – funkcja, która dla podanej liczby neuronów w warstwie wejściowej *Ni*, dla podanej liczby neuronów w warstwie wyjściowej *No* oraz dla podanej liczby warstw ukrytych *H* zwraca krotkę określającą liczby neuronów w poszczególnych warstwach ukrytych (wszystkie poza wejściową)

r() – funkcja pomocnicza, z której korzysta funkcja *NHN()*

create_model() – funkcja, która na podstawie liczby cech obserwacji (równej *Ni*) i liczby zwracanych przez sieć wartości (równej *No*) zwraca model typu *Sequential* o zadanej głębokości *depth* i funkcji aktywacji (na wyjściu każdej warstwy ukrytej poza ostatnią wyjściową)

zwroc_dane_mnist() – funkcja, która zwraca w formie dwóch krotek dane trenujące (*x_train*, *y_train*) i testowe (*x_test*, *y_test*) będące zbiorem 70000 etykietowanych obrazów reprezentujących cyfry od 0 do 9 napisane ręcznie (po 7000 na każdą klasę). Dane zostają w ten sposób podzielone na trenujące i testowe w stosunku 6 : 1.

rysuj_n_na_m_obserwacji() – funkcja, która wyświetla $n \cdot m$ obrazów reprezentujących obserwacje z podanej na wejściu tablicy. Obrazy są wyświetlane w postaci macierzy $n \times m$.

Na wstępie zaimportuj wszystkie składowe modułu *utils*:

```
from utils import *
```

Następnie załaduj dane trenujące i testowe:

```
(x_train, y_train), (x_test, y_test) = zwroc_dane_mnist()
```

W wyświetl ich kształty.

Następnie wyświetl przykładowe 25 obserwacji (w postaci macierzy obrazów 5 x 5):

```
rysuj_n_na_m_obserwacji(5, 5, x_train)
```

Ponieważ na wyjściu sieci będziemy chcieli dla każdej podanej na wejściu obserwacji zwrócić tablicę dziesięciu liczb – prawdopodobieństw przynależności tej obserwacji do poszczególnych klas (od 0 do 9) musimy jako funkcję aktywacji w ostatniej warstwie ustawić *softmax*. Ponadto należy zmienić format etykiet danych z jednocyfrowych (od 0 do 9) na kodowanie gorąco-jedynkowe.

Przykład:

- Dla etykiety 5 uzyskamy reprezentację:

```
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
```

- Dla etykiety 0 uzyskamy reprezentację:

```
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Aby przekonwertować etykiety do postaci gorąco-jedynkowej można zastosować instrukcje:

```
y_train = tf.one_hot(y_train, depth=output_size).numpy()
```

```
y_test = tf.one_hot(y_test, depth=output_size).numpy()
```

Aby nasze obserwacje mogły być danymi wejściowymi w procesie uczenia sieci sekwencyjnej należy przekonwertować je z postaci 2D (28 x 28) na 1D (1 x 784):

```
x_train = x_train.reshape((-1, input_size)) / 255.
```

```
x_test = x_test.reshape((-1, input_size)) / 255.
```

gdzie:

- *input_size* – rozmiar pojedynczej obserwacji po spłaszczeniu (784)
- dzielenie przez 255.0 jest związane z potrzebą normalizacji danych (sprowadzenie poziomów jasności pikseli do zakresu od 0.0 do 1.0)

W kolejnym kroku należy utworzyć funkcję celu, za pomocą której będziemy optymalizować architekturę budowanego modelu pod kątem uzyskiwanej przez niego dokładności. Będziemy dobierać rodzaj funkcji aktywacji oraz głębokość sieci (liczbę jej warstw ukrytych – wszystkie warstwy poza wejściową).

Wskazówki implementacyjne:

- Aby zdefiniować zbiór nazw funkcji aktywacji, z którego będziemy w kolejnych iteracjach wybierać jedną z nich możemy skorzystać z funkcji `suggest_categorical()` wywołanej na bazie obiektu `trial`:

```
activation_func = trial.suggest_categorical("activation_func",
["relu", "tanh", "sigmoid", "swish"])
```

- Wewnątrz funkcji celu należy po dobraniu głębokości sieci i rodzaju funkcji aktywacji utworzyć model za pomocą funkcji `create_model()` z modułu `utils` wykorzystując globalne zmienne `input_size`, `output_size`.
- Tym razem należy dokonać kompilacji modelu z wykorzystaniem optymalizatora `sgd` (stochastyczny spadek wzdłuż gradientu) i funkcji błędu typu *kategoryczna entropia krzyżowa*:

```
model.compile(optimizer="sgd", loss=tf.losses.CategoricalCrossentropy())
```

- Uczenia modelu dokonujemy za pomocą funkcji `fit()` wywołanej na jego rzecz:

```
model.fit(x_train, y_train, verbose=0)
```

- Funkcja celu powinna zwracać dokładność modelu obliczoną za pomocą funkcji `accuracy_score()` z modułu `sklearn.metrics`:

```
return accuracy_score(tf.argmax(y_test, 1), tf.argmax(model.predict(x_test), 1))
```

Gdzie funkcja `argmax()` z modułu `tensorflow` została użyta w celu znalezienia indeksu, pod którym w wektorach gorąco-jedynkowych etykiet znajdują się jedynki oraz w celu znalezienia indeksu, pod którym w wektorach predykcji znajdują się największe wartości. W przypadku wektorów predykcji funkcja ta zwróci po prostu oznaczenie cyfry, którą do tej obserwacji przypisał model w fazie przewidywania (inferencji).

W kolejnej fazie należy przeprowadzić proces optymalizacji funkcji celu w 25 iteracjach w kierunku poszukiwania maksimum. Następnie należy wyświetlić głębokość i funkcję aktywacji najlepszej znalezionej sieci oraz wartość dokładności osiąganą przez tę sieć.

12. Analiza działania nauczonej sieci

W tej części należy utworzyć model (z wykorzystaniem funkcji `create_model()`) o liczbie warstw i funkcji aktywacji znalezionych w zadaniu 11. Następnie należy przeprowadzić uczenie tego modelu w oparciu o dane trenujące `x_train` i `y_train`. Następnie dla nauczonego modelu należy przeprowadzić analizę uzyskiwanych wyników:

- Parametry dokładność, precyzja i czułość – należy wykorzystać funkcję `classification_report()` z modułu `sklearn.metrics`:

```
classification_report(y_test, y_pred_classes)
```

gdzie `y_pred_classes` oznacza predykcje wykonywane przez model dla obserwacji testowych

Poniżej wyjaśniono krótko znaczenie poszczególnych metryk:

Dokładność to stosunek liczby poprawnie zaklasyfikowanych obrazów do całkowitej liczby obrazów; w przypadku MNIST oznacza to, ile z 10 możliwych cyfr model rozpoznał prawidłowo na podstawie całego zbioru testowego.

Precyzja to odsetek przypadków, które zostały zaklasyfikowane jako dana cyfra i rzeczywiście nią były; na przykład: jeśli model przewidział cyfrę '3' 100 razy, ale tylko 80 z nich faktycznie było '3', to precision dla klasy '3' wynosi 80%.

Czułość (recall) to odsetek rzeczywistych przypadków danej klasy, które model prawidłowo rozpoznał. Na przykład: jeśli w zbiorze testowym było 120 zdjęć cyfry '7', a model rozpoznał 90 z nich, to *czułość* dla klasy '7' wynosi 75%.

- Macierz pomyłek – graficzna reprezentacja prezentująca dla każdej pary klas (a, b), gdzie $a, b \in \{0,1,2, \dots, 9\}$, ile razy klasa a została rozpoznana jako klasa b (i odwrotnie). W celu wygenerowania macierzy pomyłek można wykorzystać poniższy kod:

```
from sklearn.metrics import confusion_matrix

import seaborn as sns

import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_classes)

sns.heatmap(cm, annot=True, fmt='d')

plt.xlabel("Przewidywane")

plt.ylabel("Rzeczywiste")

plt.show()
```